

Software Factories, Light and Dark

JULY 20, 2026

A software factory is harnessing loops at scale. You can run the loop with humans in it (light factory): trading judgment and concentration against speed and breakage. Or you can ignore the humans (dark factory) and let those agents scope, build and ship code, without anyone really reading the details. But if people stop reading, they'll stop understanding your software. Your hardest job now is knowing which checks to build and how much autonomy to delegate.

This idea of the software factory is a term that dates back to [Bob Bemer's](#) paper, "The economics of program production," given in 1968. For half a century, many have dreamed of a world in which software is a repeatable and instrumentable production process (analogous to stamping out car parts in a factory) rather than the isolated craft of individuals. Historically, this dream has generally (although not universally) fallen flat, in part because of the difficulty of stamping out ideas.

But in the last two years, things have changed dramatically enough that now it makes sense to take a fresh look at the old dream. And since some subtleties can easily be glossed over, it's worthwhile to be somewhat precise about exactly what is really new and different, and what may be recurring traps, dressed up as new opportunities.

Dex Horthy, co-founder of HumanLayer recently gave a great talk at AI Engineer World's Fair called "Harness Engineering is not Enough: Why Software Factories Fail." worth checking out on this topic.

The loop is the atom. The factory is the loop at scale.

Structure is everything, and it all starts with small units. The whole stack is really three concepts layered on top of each other: the loop, the harness, and the factory.

A loop is one agent doing a single job on repeat: gather context, take an action, check the result, and go again until some condition is met. It is the smallest unit of agentic work, and everything above it is just loops stacked on loops.

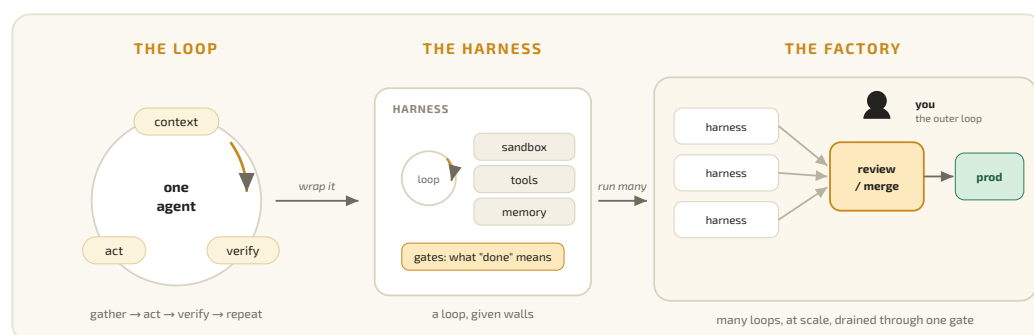
The point of **loop engineering** is that you stop prompting the agent turn by turn and instead design the small system that prompts it for you.

A harness is the walls around a loop: the sandbox it runs in, the tools it can reach, the memory that survives between runs, and the gates that decide what “done” means. The loop is the behavior; the harness is the environment that behavior runs inside.

Hand a raw model no harness and it will happily spin forever. The harness is everything around it that makes it useful and safe to run.

A software factory is many harnessed loops running at once, fed by a queue of work and drained through a review gate into production, with humans owning the whole thing from above. It is not a bigger agent; it is an org chart made of loops.

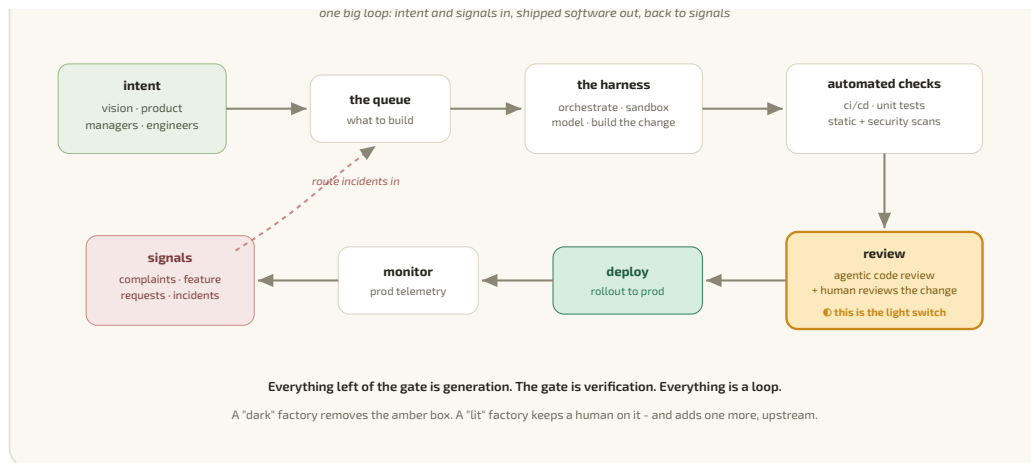
The final paradigm shift is moving from writing code to building and running **the factory** that writes it. The unit of work shifts up a level, to the loop, the harness, and the flow between them, rather than the individual code diff.



Loop → harness → factory. A factory isn't a smarter agent; it's many harnessed loops feeding one review gate, with a human owning the outer loop.

The factory, drawn

The central slide Dex spent most time on was brilliant because it's a clarifying wiring diagram that visualizes what otherwise is an obvious loop. Here's my take on it:



The factory is a closed loop: intent and production signals feed a queue, the harness builds, automated checks and review gate it, deploy ships it, monitoring turns prod back into signals.

Intent flows from the vision of engineering leadership, and directly from engineers, into a queue of things to be done. Signals driven by incidents and user requests drive the same queue. The harness is just the thing that picks an item from the queue and builds a change for it. Beyond the harness, we can see all the automated checks required to make changes safe enough to let into production. These automated checks run at once, effortlessly, without any conscious involvement from engineers, thanks to CI, tests, static analysis, and scanning of all kinds. The only decision point here is the review gate. After approval, changes are deployed and monitored in production, with monitoring data feeding back into the signals that kicked the loop into motion to begin with.

By and large, every box in this diagram is almost zero cost: generation, tests, scanning. They all run at scale for negligible cost. There is only one expensive box that proves stubbornly resistant to scaling, and that's the review gate. That shiny amber box is "judgment", and where the crux of the argument about whether we can make development faster and more frequent resides.

Why we call it "dark"

A dark factory runs with the lights physically off, because the only things on the floor are machines and machines don't need light to see. A dark software factory is the same move: code ships that no human has read, verified only by other machines.

The image is borrowed from manufacturing. Its origins are physical rather than digital, rooted in facilities where the lights are turned off and the work is carried out by robots. [FANUC in Japan has been running lights-out factories of this sort since 2001](#); Xiaomi, in 2024, opened a heavily automated dark factory of its own. What these

have in common is a product assembled and shipped without a single human having read any of it. The “dark” comes in when that act of reading is removed from the process.

I’m not borrowing the concept for its vibe or as an insult. For all its creepy buzz, “dark” here is a simple physical claim: the original factory floor, but without light. In software, the floor is the diff. Whoever wrote the diff, whoever reviewed it, whoever shipped it, those humans are gone, and what remains is a diff verified only by the machines that built it.

This is a surprisingly easy thing to do, at least at first. It’s easy because that missing review step gets in the way of everything. Its absence makes your perception of your team’s vertical throughput seem suddenly and radically higher. It feels as if you’ve broken the sound barrier. For all its apparent ease, it’s harder than it seems to survive those dark workflows, with all their buried costs.

Harness engineering is not enough

The harness of orchestration, sandboxed prototyping, and tool calling as models interact with the world and each other will become increasingly powerful and effective. However, there’s an inherent in-model failure in trying to keep up with codebase quality over the long game and through additive changes, and I think there’s good reason to believe that models alone will ultimately lose that battle against [comprehension debt](#).

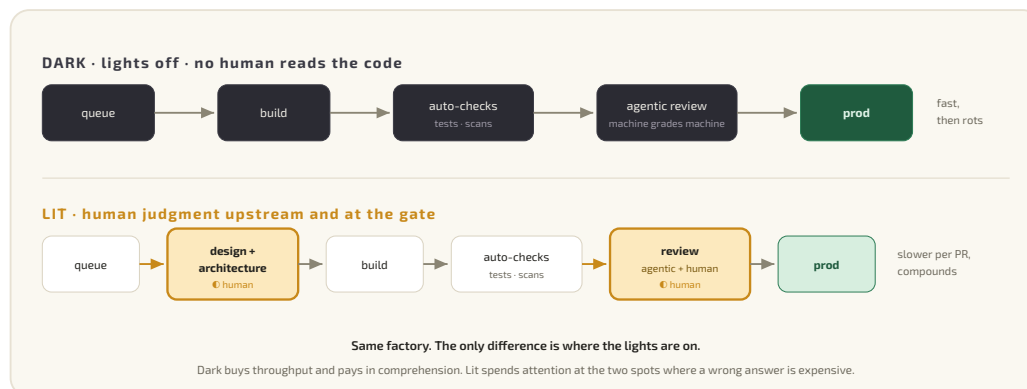
Comprehension debt is the widening gap between how much code exists and how much any human still understands. A dark factory doesn’t pay it down; it takes it on as fast as it can, with the tests green the whole way.

This is an important distinction because models do well at some tasks. But for anything that isn’t an immediate change to a small part of a codebase, especially in a complex brownfield system, model-only automated coding faces an insurmountable obstacle. Weekend toys and side projects are alike in that a few months of development cycles is usually enough to get things in working order, or at least close enough. But an enterprise system that has been under development for a decade or more is a different beast; it has to be maintained, in a professional environment at a professional pace. Three to six months into a project, you’re already drowning in unread code. That kind of environment, and especially the constraints enforced by production code, would make even a powerful agent do poorly, all of it in contrast to

the vibe-coding enjoyed by developers working on weekend toys.

Dex reports from experience that this is a major failure, so much so that it required painstaking manual debugging to pinpoint. This came from running a fully automated code factory for about four months, during which no human looked at the code that was written. Underlying the experience is a tradeoff between two conflicting metrics. One is maximizing token utilization, the number we currently treat as progress. The other, which it quietly minimizes, is the amount of the system any human participant still understands at any moment.

Where the dark factory truly shines is in its ability to burn through pristine code while the tests stay green. The ultimate reckoning, when it comes, will not be a dramatic “it all goes sideways” moment. It will be quiet and late.



Dark and lit are the same pipeline with the lights in different places. The lit version doesn't just re-add review at the end - it moves human judgment upstream to design and architecture, too.

The bottleneck was never generation

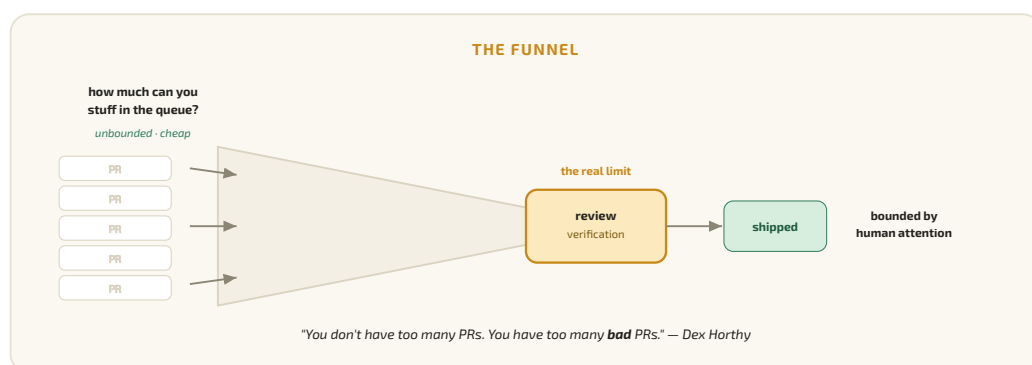
The fundamental constraint in a software factory isn't how much code we can churn out: it's how quickly we can verify it.

Back pressure is the rule that you can only hand a loop as much autonomy as you can cheaply and reliably verify, and not one inch more. Verification, not generation, is the real constraint on a factory.

Because unbounded generation capacity is in perpetual tension with the finite, non-scaling resource of human attention, the core problem is the gap between cheap generation and bounded review. Look at the funnel: as long as the neck representing verification doesn't widen, it's going to back up. As Dex points out, volume alone isn't the problem: what we're really suffering from is a surplus of bad PRs. When you've got high volume without trustworthy gates, manufactured defects are unavoidable.

This is just back pressure again: autonomy can't expand beyond what can be cheaply and reliably verified.

The second-order problem is why improving the model shouldn't automatically close the gap between what it can generate and what can be verified. Training on well-architected systems is an arguably more difficult proposition than passing simple tests: remember, the cost functions measuring architectural excellence aren't measured in seconds or even minutes, but in months and years. Tidy gradients are functionally impossible to compute, so a system expecting crisp, instant evaluation of complex design decisions isn't going to be trained on good examples.



Generation is a wide mouth; verification is the narrow neck. Speeding up the mouth just deepens the pile at the neck.

Turning the lights back on

A lit factory is the same pipeline with the lights left on where judgment lives. The agents still do most of the building, but a human reads what comes out before it ships, and the lights stay on wherever a wrong call is expensive.

The lit version doesn't tack review onto the end but moves the point of human judgment upstream, to the product, the design, and the architecture before an agent starts a loop.

One great thing about that upfront hour is that it leads to fewer implementation hours. It turns a long, frustrating code review into a quick read of a two-hundred-line plan. You get to review a decision before it's built, so later you aren't chasing through two thousand lines of generated code to find out what the decision even was. Some decisions are expensive and long-lived enough that you'd want a person in on them early, before the cost compounds. Of course, there are still times you look at diffs, even when you've spent time up front.

You might be thinking that all sounds unglamorous. You're right. The safety net is

made up of perfectly ordinary architectural practices we've always known about and mostly ignored: good types and method signatures so that mistakes are caught by the compiler instead of in production; test seams where we can pin behavior and make change observable; laying out the code so the next reader, human or model, knows where to find the thing they care about; keeping call stacks short and legible; keeping component boundaries well defined so a change doesn't have a huge blast radius; and dependency injection so we can swap out one piece for another. None of it is new. We've always said we care about good architecture. But now that we're using automated coding agents, that architecture is finally doing a second job as a cheap and hard-to-fake safety net against the mistakes the agent will make.

That safety net has to live outside the model, because the model won't supply it. The coding agents that feel most capable, Claude Code and Codex among them, are reinforcement-trained against their own harness and tools: fluent with all the tools and idioms of the trade, but not with things like long-term maintainability. The deliberate architecture we've always talked about is the tool that catches that debt, and the investment we make in it is us buying back our autonomy. Put that together with safe infrastructure, and there are some tight, low-risk loops you can run unattended. Horthy described one in a recent post: a nightly GitHub Actions cron that fixes exactly one anti-pattern, a lint violation or a needlessly optional prop, commits, and opens one small pull request, all on its own, so the team wakes up to a slightly better codebase and a diff short enough to read. But for loops with high enough stakes, you don't want to risk waking up to a broken auth system, billing engine, or public API contract. Keep the lights on there, and trust that a person with judgment and a real working knowledge of the system will catch the mistake.

What earns a loop the dark

This rule applies whether you call it back pressure, verification, or the light switch.

A loop can earn itself fully automated status only if the check is cheap, runs at high frequency, and relies on something that can't be easily faked out. Green-or-red oracles, type gates, property tests, and a review agent coupled with a real rubric all fit. You also need the oracle to answer immediately and not drift over time. When done can be proven not just by you but by a machine, you've reached automation.

Short loops are easier to verify than long ones. [Dex's rule of thumb](#): an agent holds up for three to ten steps, then starts losing the thread past twenty. The reason is

context accumulation, the more the agent drags along, the more likely it is to wander off. When a loop is short, verifying it is cheap. Sprawling loops hide mistakes in the corners, which is another way of saying they never earned lights-out status.

Keeping the lights on is the opposite case. A loop needs to be reviewed if a wrong answer is expensive and only a person can catch it. Subtle production bugs that can't be caught by tests, large blast radii, and a decision that's going to shape the work of a year or more all qualify. In those cases, [your attention is the actual product](#), the costly, essential one.

The danger is forgetting to flip each switch and just setting all of them to the same mode. All dark, and you're stuck tearing everything down four months later. All lit, and no one can get reviews done in time and you're stuck in a gigantic bottleneck. The hard, skilled job is deciding where to put each switch.

Loops, graphs or state machines?

When you hand an agent a task, you're probably going to build a graph around it, whether you call that graph a finite state machine or a set of conditionally-linked service calls. It's a framing where the software isn't just following some abstract rules but a structured workflow: every node is an explicit step, and every edge between nodes is an explicit condition. That sounds like a lot of structure, but most of it is already there in any software, since any code can be expressed as a control-flow graph. So the only real novelty is that an agent insisting on autonomy is really just walking around a particular graph, and its freedom is constrained to the inside of a node. And here's the part people forget, which [Dex wrote down](#) a year ago: software was always going to have that structure. There's a reason we used to draw programs as flow charts. The genuinely new move was trying to throw the diagram away, leaning on a loop where the model picks the path tool call by tool call, until it declares itself done. That felt like liberation, right up until it met a ten-year-old codebase, and the discipline everyone is now rediscovering, owning your control flow, is really just walking the graph back around the loop. So the question of whether we should shift from loops back to graphs is almost an admission that we needed the flowchart all along.

Here's what it looks like in practice. Take a bug to fix. As a pure loop, you sit down and think: figure out what's wrong, change some code, run the tests, see what happens, and if that round doesn't kill the run, loop back and start again. The whole

journey is decided as you go, which problem you chase, the exact code you change, which tests you run and in what order, whether you run tests at all, and whether you try again or declare victory. As a graph, the first thing you do is map out what should happen. Reproduce the bug or go ask for more information, find the cause, try a fix, run the tests, and let a failing run route back to the fix while a passing one goes on to review, where only an approval reaches done. The agent is still clever inside each box; it just can't wander off the paths you sanctioned. Santi laid this out with a diagram that makes the difference obvious.

The real appeal of that graph, of course, is that it's back pressure drawn as a diagram. You give up some of the agent's freedom and get mandatory checks and legible failure points in return, so when a run dies you can point at the node that killed it. It's the same instinct behind Dex's blunt line that most so-called agents aren't very agentic at all, "mostly deterministic code, with LLM steps sprinkled in at just the right points." And this isn't just an artifact of how people happen to be building things right now: you can see the pattern in LangGraph and LlamaIndex Workflows, in Jerry Liu's hybrid workflow-graph-over-agents with an outer loop that grows parts of the graph as it runs, and in David Kourshid's reminder that this is really just state machines and the actor model turning up in new clothes.

One clarification, because the term is badly overloaded: when I keep calling this a graph, I don't mean a knowledge graph. I mean a predefined directed graph of how the work should flow, conditional edges and all, giving the loop a shape you can actually trust.

Where the human actually goes

Notice that the person never left the factory. They moved.

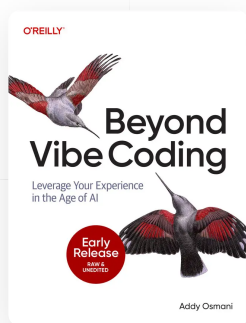
I think engineers need to increasingly own the [outer loop](#). The agents can investigate a bug, write up the diagnosis, implement a fix, run the tests, and write up a report. That's the execution of the inner loop, and they can do it as efficiently as anyone. But that was never the job. The bits you own are what I'd call the outer loop: decide whether it's the right way to address the problem, verify that the diagnosis and implementation are sound, approve the change, and carry the consequences of being wrong. The boundary between the two loops is evidence, the diffs, the tests, the logs, and a brief explanation that connects them. Types, seams, and rubrics make it possible to oversee all this without doing a lot of work for every change.

I think it's useful to put it this way: you're not down on the line writing changes any more; you're up at the end of the production line designing it and guarding the gate. There's a lot you can do to make the model better and the harness more capable, but I've observed that identifying problems that are expensive in the long term is not typically something you can automate away. The core thing that's still the job is to exercise human judgment better than any flow of paper and computing power.

Robots are fine operating in the dark, but humans need to see what they're doing. If everything on the factory floor is dark, and you can't see anything, and you can't even find the light switch, that's where the danger is.

Pangram [scored](#) this article as 100% human written.

ENJOYED THIS?



Go deeper in *Beyond Vibe Coding*

My O'Reilly book on AI-assisted and agentic engineering: specs, harnesses, evals, context, and shipping production-grade software with AI.

[Read the book](#)



Addy Osmani is an engineering and evangelism leader who spent over 14 years at Google leading developer experience across Chrome and, in recent years, AI (Gemini, coding agents, and agentic engineering), most recently as a Director at Google Cloud AI.

 [Tweet](#)

 [Bluesky](#)

 [Mastodon](#)

 [Threads](#)

 [LinkedIn](#)

 [Share](#)

Want more? [Subscribe to my free newsletter:](#)

example@gmail.com

Subscribe

Disclaimer: The views and opinions expressed on this site are my own and do not necessarily reflect the views, positions, or strategies of Google or any of its affiliates.

© Copyright 2026 Addy Osmani
